
Bridging RTL and Theorem Proving: LLM-Assisted Translation from Verilog to Lean

SooHyuk Cho

Department of Electrical and Computer Engineering
Princeton University
Princeton, NJ
soohyuk.cho@princeton.edu

Yu-Wei Fan

Department of Electrical and Computer Engineering
Princeton University
Princeton, NJ
yf9172@princeton.edu

1 Introduction

Lean [1] is an interactive theorem prover based on dependent type theory that has become popular for building machine-checked proofs and verifiably correct software, supported by a growing ecosystem (Lean 4 [1], mathlib [2], CSlib [3]) and strong automation via tactics and integration with external solvers. In the LLM era, Lean’s biggest advantage is that it can act as a precise proof checker for model- or LLM-generated arguments: the system accepts only proofs that typecheck, turning “reasoning” into a verifiable artifact. However, Lean still has limited out-of-the-box support for hardware verification compared with established HDL/formal toolchains: there isn’t a mature, standardized stack for RTL semantics, bit-precise word-level reasoning, timing/clocked models, and tight integration with mainstream RTL flows (Verilog/SystemVerilog) and industrial SAT/SMT engines. Our goal is to close that gap by enabling a full Lean-based hardware verification pipeline that leverages LLMs for proof and model generation and Lean for proof checking: in the short term, we will target autoformalization from RTL into a Lean model; in the mid term, we will use LLMs to generate Lean proofs over the extracted model; and in the long term, we aim for an end-to-end, Lean-native hardware formal verification workflow from RTL to mechanically verified correctness results.

2 Project Plan

The goal of this project is to develop and evaluate a framework for translating hardware designs written in (System)Verilog into formal representations in the Lean theorem prover. The goal is to explore the feasibility of using LLMs to bridge the gap between hardware descriptions and theorem-proving environments, enabling more automated hardware verification workflows.

We propose two approaches:

1. **LLM-generated Lean translation from Verilog**
2. **AST-based translation pipeline as a fallback approach**

The first approach uses LLMs to directly translate Verilog modules and properties into Lean definitions and theorems. The second approach constructs a more structured translation pipeline based on abstract syntax trees (ASTs) extracted from the Verilog code.

The goal is to evaluate the feasibility, correctness, and reliability of these two approaches.

2.1 Approach 1: LLM-Generated Lean Translation

Our primary approach is to leverage large language models to translate Verilog code into Lean definitions and theorem statements.

Given a hardware module written in Verilog, the LLM will be prompted to generate a Lean representation capturing the behavior of the module. The translation will include:

- Lean definitions for hardware state
- transition functions describing hardware behavior
- theorem statements corresponding to hardware properties

For example, the LLM may translate a register update operation into a Lean function describing the next-state relation of the hardware system.

The workflow is illustrated below:

Verilog Module $\rightarrow$ LLM Translation $\rightarrow$ Lean Representation $\rightarrow$ Lean Proof Checking

Lean will then be used to check the correctness of the generated representations and proofs. Compilation failures or proof failures will indicate translation errors or incorrect reasoning by the LLM.

2.2 Approach 2: AST-Based Translation (Fallback)

If direct LLM-based translation proves unreliable, we will implement a structured translation pipeline based on the abstract syntax tree (AST) of the Verilog code.

The process will involve the following steps:

1. Parse Verilog code to obtain its AST using existing tools such as Yosys [4] or Surelog [5].
2. Convert the AST into an intermediate representation capturing hardware semantics.
3. Generate Lean code from the intermediate representation.
4. Optionally use LLMs to assist in generating Lean definitions or theorem statements.

The pipeline is:

Verilog $\rightarrow$ AST $\rightarrow$ IR $\rightarrow$ Lean Code Generation

This structured approach reduces reliance on LLM reasoning and ensures that the translation process preserves hardware semantics.

3 Evaluation

We will evaluate our approach using several open-source hardware modules through public repositories like OpenTitan [6]. The potential candidates we are considering now are:

- FIFO buffers
- register files
- simple arbiters
- arithmetic logic units

For each module, we will attempt to generate Lean representations and verify properties such as:

- read-after-write correctness
- ordering guarantees

- mutual exclusion
- data consistency

We will evaluate the approaches using the following metrics (subject to change):

- **Translation success rate:** percentage of Verilog modules successfully translated to Lean.
- **Compilation success:** percentage of generated Lean code that compiles.
- **Proof success rate:** percentage of properties successfully verified.
- **Manual correction effort:** amount of manual modification required.

4 Deliverables

We will provide the following deliverables in addition to the final report for reproducing our results:

1. Module-level testcases in Verilog.
2. Prompts and/or AST-based implementation for the Verilog-to-Lean translation
3. Scripts for running experiments and reproducing relevant tables/figures in the final report.

References

- [1] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science (LNAI)*, pages 625–635. Springer, Cham, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.
- [2] The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, New Orleans, LA, USA, January 2020. ACM. doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.
- [3] Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The Lean computer science library, 2026. URL <https://arxiv.org/abs/2602.04846>.
- [4] Clifford Wolf, Johann Glaser, and Johannes Kepler. Yosys-a free verilog synthesis suite. In *Proceedings of the Austrian Workshop on Microelectronics (Austrochip)*, 2013.
- [5] CHIPS Alliance and contributors. Surelog: Systemverilog 2017 pre-processor, parser, elaborator, uhdh compiler. <https://github.com/chipsalliance/Surelog>, 2026. Open-source SystemVerilog 2017 front-end producing UHDM.
- [6] lowRISC CIC. Opentitan: Open source silicon root of trust, 2026. URL <https://opentitan.org/>. Accessed: 2026-03-05.